Dockerizing a Python App

A Complete Step-by-Step Exercise for Beginners

This document provides a hands-on exercise to teach you how to containerize a simple Python web application using Docker. By the end of this exercise, you will have created a Python app, written a Dockerfile, built a Docker image, and run it as a container.

> **Prerequisites:** Before you begin, ensure you have **Docker Desktop** installed and running on your machine, and a basic understanding of using the command line (Terminal or Command Prompt).

Step 1: Create the Python Application

First, we need a Python application to containerize. We will build a simple web server using the Flask framework.

1. Create a new directory for your project and navigate into it:

```
mkdir python-docker-exercise
cd python-docker-exercise
```

2. Create a file named `app.py` and add the following Python code:

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Dockerized Python! The app is running successfully."

if __name__ == "__main__":
    # Bind to 0.0.0.0 to make it accessible outside the container
    app.run(host="0.0.0.0", port=5000)
```

3. Create a file named `requirements.txt` to list our dependencies:

```
Flask==2.3.2
Werkzeug==2.3.6
```

Step 2: Create the Dockerfile

The `Dockerfile` is a text document that contains all the commands needed to assemble an image. Create a file named strictly `Dockerfile` (no file extension) in the same directory and add the following:

```
# 1. Use the official lightweight Python image
FROM python:3.9-slim

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy the dependencies file to the working directory
COPY requirements.txt .

# 4. Install the Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the rest of the application code
COPY app.py .

# 6. Expose the port the app runs on
EXPOSE 5000

# 7. Define the command to run the application
CMD ["python", "app.py"]
```

Command Explanations

Here is a detailed breakdown of what each command in the Dockerfile does:

Command	Detailed Explanation
`FROM python:3.9-slim`	Defines the **base image**. Instead of installing Python from scratch on a blank OS, we start with an official image that already has Python 3.9 installed. The `-slim` tag means it's a smaller, stripped-down version of Debian Linux, making our final image smaller and faster to download.
`WORKDIR /app`	Sets the **working directory** inside the container. Any subsequent `COPY`, `RUN`, or `CMD` commands will be executed in this folder. If `/app` doesn't exist, Docker creates it.
`COPY requirements.txt .`	Copies the `requirements.txt` file from your local computer (the host) into the container's current working directory (`/app`). The dot `.` represents the current directory.
`RUN pip install ...`	Executes a command during the image build process. Here, it installs the Flask framework listed in our requirements file. `--no-cache-dir` prevents pip from storing downloaded packages, keeping the image size small.
`COPY app.py .`	Copies your Python script into the container. *Note: We copy requirements and install them BEFORE copying the app code. This is an optimization trick—if we change the code but not the requirements, Docker doesn't have to reinstall everything!*
`EXPOSE 5000`	This is a documentation feature. It tells Docker that the container will listen on port 5000 at runtime. (It does **not** actually publish the port to your host machine; that happens in Step 4).
`CMD ["python", "app.py"]`	Specifies the **default command** to execute when the container starts. Unlike `RUN` (which executes during the build), `CMD` executes only when you actually run the image.

Step 3: Build the Docker Image

Now, we will read the Dockerfile and build a reusable Docker image. Run the following command in your terminal:

```
docker build -t my-python-app .
```

Command Explanations

Argument	Detailed Explanation
`docker build`	The base command to build an image from a Dockerfile.
`-t my-python-app`	The `-t` (tag) flag lets you name your image so you can find it later easily. Here, we name it `my-python-app`.
`.` (The Dot)	Tells Docker the **build context** is the current directory. Docker looks for the `Dockerfile` and the files to `COPY` right here. Do not forget the dot!

Step 4: Run the Docker Container

An image is a static blueprint. To actually execute your application, you need to run it as a container.

```
docker run -d -p 5050:5000 --name my-running-app my-python-app
```

Command Explanations

Argument	Detailed Explanation
`docker run`	The command to create and start a container from an image.
`-d`	Runs the container in **detached mode** (in the background). It gives you your terminal prompt back immediately instead of locking it up with application logs.
`-p 5050:5000`	**Port Mapping** (`HostPort:ContainerPort`). It forwards traffic from port `5050` on your local computer to port `5000` inside the container (where Flask is listening).
`--name my-running-app`	Assigns a custom, memorable name to the container. If omitted, Docker generates a random funny name (like `sleepy_einstein`).
`my-python-app`	The name of the image we built in Step 3 to base this container on.

Step 5: Verify it Works

Your Python application is now running inside an isolated Docker container!

Open your favorite web browser and navigate to:

```
http://localhost:5050
```

You should see a plain text page displaying: *"Hello from Dockerized Python! The app is running successfully."*

Step 6: Stop and Clean Up

Once you are done testing, you can stop the container. Since we named it, stopping it is easy:

```
# Stop the running container
docker stop my-running-app

# Remove the stopped container
docker rm my-running-app
```

> **Congratulations!** You have successfully containerized a Python web application, built a custom Docker image from scratch, and managed a running container.